

GitHub Repo: https://github.com/lisha080/CSE464_2026_Iseekalr

1. In the new refactor branch: https://github.com/lisha080/CSE464_2026_Iseekalr/tree/refactor

Refactoring 1: Rename I renamed dfsHelper to dfsRecursive. The old name didn't say anything about what the method does, "helper" is pretty vague. dfsRecursive makes it immediately clear that this is the recursive step of the DFS algorithm.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/be175cb902ecc41a4530f4f12f5942ad2f6ec9be

Refactoring 2: Extract Method I extracted the duplicate edge check out of addEdge into its own method called edgeExists. addEdge was doing two things at once: checking if an edge already exists and then adding it. Giving the check its own method makes addEdge easier to read and gives the check a clear, descriptive name.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/e291093f80109fa8105d263d371939742fddf2e7

Refactoring 3: Extract Variable I extracted nodes.size() and edges.size() into named variables nodeCount and edgeCount inside toString(). Having method calls inline inside string concatenation made the code harder to read at first glance. Naming the values makes it more immediately obvious.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/6878226248c0961f9e2f581481b68d5dc199ac7f

Refactoring 4: Extract Method I extracted the iterator loop that removes edges connected to a deleted node into its own method called removeEdgesContaining. removeNode was doing two separate things: removing the node itself and cleaning up its edges. Separating them makes both operations easier to understand.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/6877604517613f264d6afecdcb8464a8f0c8e0df

Refactoring 5: Extract Method I extracted the validation loop in removeNodes into its own method called validateNodesExist. The check that all nodes exist before any are removed is a separate thing from the actual removal. Pulling it out makes removeNodes cleaner and makes the validation step reusable if needed later.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/35a2b5c83e0c01e1e637d8518490b1afc63d8e2b

2. Template Pattern: I created an abstract base class GraphSearchTemplate that defines the skeleton of the search algorithm. The common step of checking if src and dst exist in the graph lives in the base class. The part that differs between algorithms, executeSearch(), is abstract and implemented by each subclass.

BFSSearch extends GraphSearchTemplate and implements executeSearch using a Queue. DFSSearch extends GraphSearchTemplate and implements executeSearch using recursion.

Neither subclass needs to worry about the null check because that's just handled once in the base class.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/75d1f406375f4f605c1e460137681a7a74d31a88

3. Strategy Pattern

I created a SearchStrategy interface with one method: search(src, dst, nodes, edges). Both BFSSearch and DFSSearch implement this interface in addition to extending the template. In Graph.java, graphSearch() declares the searcher as type SearchStrategy, picks the right concrete class based on the algo parameter, and calls search(). This means Graph.java depends on the interface, not the concrete classes directly, so adding a new algorithm later just requires implementing the interface.

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/9e6b0a7fac3c63a94319da5ee4886e290e8264e6

4. **Random walk** picks a random unvisited neighbor at each step until it finds the destination or runs out of options. It uses both the template pattern (extends GraphSearchTemplate) and the strategy pattern (implements SearchStrategy).

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/656852229ac3ab431f5579cc87da650caf73dda1

5. Pull Request:

https://github.com/lisha080/CSE464_2026_Iseekalr/pull/1

Five “reviewer” comments:



lisha080 commented [5 days ago](#)

Owner Author ...

GraphTest currently does not test BFS/DFS/Random strategy behavior

😊



lisha080 commented [5 days ago](#)

Owner Author ...

For Random Walk Search, you allowed the revisiting of nodes (with a step limit) there are higher chances of finding the target node rather than null

😊



lisha080 commented [5 days ago](#)

Owner Author ...

In Graph.java if/else strategy selection works, but adding new algorithms requires modifying this method.

😊



lisha080 commented [5 days ago](#)

Owner Author ...

BFS/DFS do not print the search logic/inner steps whereas the Random Walk Search does. Visually, it does not look as uniform because of this.

😊



lisha080 commented [5 days ago](#)

Owner Author ...

IN Graph.java, the else branch in graphSearch: The else defaults to DFS. If someone adds a new algorithm value later and forgets to handle it here, it will just run DFS without any warning. An explicit else that throws an `IllegalArgumentException` would make this much safer.

😊

Code changes/addresses to the comments:

```
@Test
void testBFSsearch() throws Exception {
    Graph g = new Graph();
    g.parseGraph(filepath: "input.dot");
    Path p = g.graphSearch(src: "a", dst: "h", Graph.Algorithm.BFS);
    assertNotNull(p);
    assertTrue(p.toString().startsWith("a"));
    assertTrue(p.toString().endsWith("h"));
}

@Test
void testDFSsearch() throws Exception {
    Graph g = new Graph();
    g.parseGraph(filepath: "input.dot");
    Path p = g.graphSearch(src: "a", dst: "h", Graph.Algorithm.DFS);
    assertNotNull(p);
    assertTrue(p.toString().startsWith("a"));
    assertTrue(p.toString().endsWith("h"));
}

@Test
void testRandomWalkSearch() throws Exception {
    Graph g = new Graph();
    g.parseGraph(filepath: "input.dot");
    boolean foundPath = false;
    for (int i = 0; i < 20; i++) {
        Path p = g.graphSearch(src: "a", dst: "c", Graph.Algorithm.RANDOM);
        if (p != null) {
            foundPath = true;
            break;
        }
    }
    assertTrue(foundPath);
}
```

1:

2: Looking at the assignment instructions, the examples seem to avoid revisiting nodes. Also, letting it revisit nodes means it could get into loops (with the limit of 1000) and not terminate for a long time. The current behavior is a design decision.

```

public Path graphSearch(String src, String dst, Algorithm algo) {
    SearchStrategy searcher;
    if (algo == Algorithm.BFS) {
        searcher = new BFSSearch();
    } else if (algo == Algorithm.DFS) {
        searcher = new DFSSearch();
    } else if (algo == Algorithm.RANDOM) {
        searcher = new RandomWalkSearch();
    } else {
        throw new IllegalArgumentException("Unknown algorithm: " + algo);
    }
    return searcher.search(src, dst, nodes, edges);
}

```

3 & 5.

4. `System.out.println("visiting " + new Path(path));` was added to the while loop inside both `BFSSearch.java` and `DFSSearch.java`

Commit Link:

https://github.com/lisha080/CSE464_2026_Iseekalr/commit/03842fd29f1015161512f06f9103c808eda1fd45

Pull Request link:

https://github.com/lisha080/CSE464_2026_Iseekalr/pull/2

To run all features including BFS, DFS, and Random Walk:

1. Make sure `input.dot` is in the project root
2. Run `mvn package` to build and test
3. Run `mvn exec:java -Dexec.mainClass="CSE464ProjectP1.Main"` to see the output

The program will automatically run all features in order including BFS from a to h, DFS from a to h, and 5 random walk searches from a to h. The output for each will print to the console.

```

≡ input.dot
1  digraph {
2  a -> b;
3  b -> c;
4  c -> d;
5  d -> a;
6  a -> e;
7  e -> f;
8  e -> g;
9  f -> h;
10 g -> h;
11 }

```

The code below is in Main.java to find the path from “a” to “h” in the input.dot file above. To find the path from or to a different node we simply change “a” or “h” to different ones.

BFS:

```
// Feature 8 – BFS search
System.out.println("=== Feature 8: BFS Graph Search ===");
Graph graph2 = new Graph();
graph2.parseGraph(filepath: "input.dot");
Path bfsPath = graph2.graphSearch(src: "a", dst: "h", Graph.Algorithm.BFS);
System.out.println("BFS path from a to h: " + bfsPath);
System.out.println();
```

BFS Output:

```
=== Feature 8: BFS Graph Search ===
visiting a
visiting a -> b
visiting a -> e
visiting a -> b -> c
visiting a -> e -> f
visiting a -> e -> g
visiting a -> b -> c -> d
visiting a -> e -> f -> h
BFS path from a to h: a -> e -> f -> h
```

DFS:

```
// Feature 9 – DFS search
System.out.println("=== Feature 9: DFS Graph Search ===");
Path dfsPath = graph2.graphSearch(src: "a", dst: "h", Graph.Algorithm.DFS);
System.out.println("DFS path from a to h: " + dfsPath);
System.out.println();
```

DFS Output:

```
=== Feature 9: DFS Graph Search ===
visiting a
visiting a -> b
visiting a -> b -> c
visiting a -> b -> c -> d
visiting a -> e
visiting a -> e -> f
visiting a -> e -> f -> h
DFS path from a to h: a -> e -> f -> h
```

Random Walk:

```
// Feature 10 - Random Walk Search
System.out.println("=== Feature 10: Random Walk Search ===");
Graph graph3 = new Graph();
graph3.parseGraph(filepath: "input.dot");

for (int i = 0; i < 5; i++) {
    System.out.println("Random Walk #" + (i + 1));
    Path randomPath = graph3.graphSearch(src: "a", dst: "h", Graph.Algorithm.RANDOM);
    if (randomPath != null) {
        System.out.println("Success: " + randomPath);
    } else {
        System.out.println("Failed: no path found");
    }
    System.out.println();
}
}
```

Random Walk Output (5 iterations but the number is adjustable with the for loop):

```
=== Feature 10: Random Walk Search ===
Random Walk #1
visiting a
visiting a -> e
visiting a -> e -> f
visiting a -> e -> f -> h
Success: a -> e -> f -> h

Random Walk #2
visiting a
visiting a -> b
visiting a -> b -> c
visiting a -> b -> c -> d
Failed: no path found

Random Walk #3
visiting a
visiting a -> e
visiting a -> e -> f
visiting a -> e -> f -> h
Success: a -> e -> f -> h

Random Walk #4
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
Success: a -> e -> g -> h

Random Walk #5
visiting a
visiting a -> b
visiting a -> b -> c
visiting a -> b -> c -> d
Failed: no path found
```

CI Workflow:

The screenshot displays the GitHub Actions interface for a repository named 'CSE464_2026_Iseekalr' by user 'lisha080'. The 'Actions' tab is selected, showing a list of workflow runs. The left sidebar contains navigation links for 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Wiki', 'Security and quality', 'Insights', and 'Settings'. Under 'Actions', there is a 'New workflow' button and a list of workflow categories: 'All workflows', 'Java CI', and 'Management'. The 'All workflows' category is selected, showing a search bar and a list of workflow runs. The list shows two runs: 'Merge pull request #1 from lisha080/refactor' and 'Refactor'. Both runs are successful (green checkmark) and show the workflow name, branch, and completion time.

Workflow	Event	Status	Branch	Actor
Merge pull request #1 from lisha080/refactor	main	Success	main	lisha080
Refactor	refactor	Success	refactor	lisha080